Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No:92301733021**

Aim: To recommend an item to the user using collaborative-based filtering approach

IDE: Google Colab

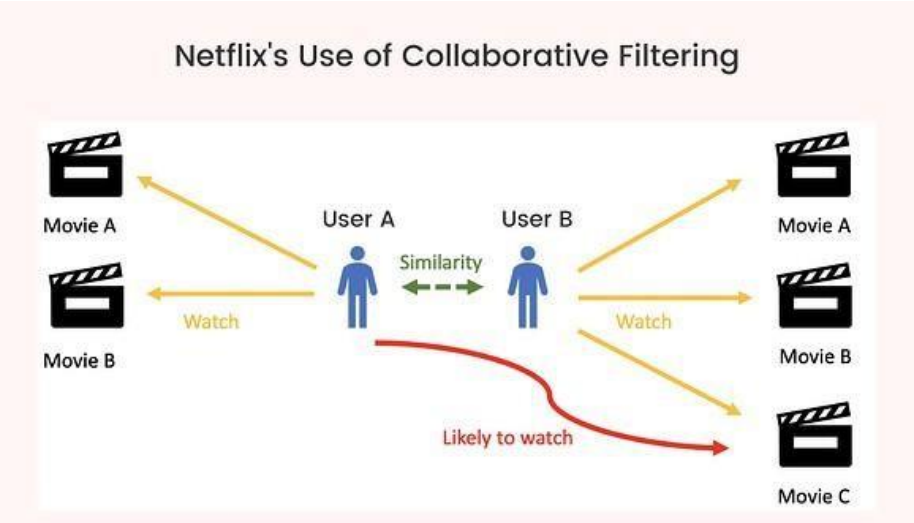
Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

In Collaborative Filtering, we tend to find similar users and recommend what similar users like. In this type of recommendation system, we don't use the features of the item to recommend it, rather we classify the users into clusters of similar types and recommend each user according to the preference of its cluster.



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No:92301733021**

Pre Lab Exercise:

1. **When can you use collaborative-based recommendation system approach?**

Collaborative filtering is used when we have data from many users, such as ratings, reviews, or purchase history. It recommends items based on what similar users like.

Example: Suggesting movies based on other users with similar tastes.

2. **Write advantages of collaborative -based filtering approach**
   - Gives personalized recommendations
   - Does not need item details (like genre or description)
   - Can suggest new or different items
   - Improves as more user data is available
3. **Write disadvantages of collaborative -based filtering approach**
   - Cold start problem (new user or new item)
   - Needs large amount of user data
   - Data sparsity issue (few ratings)
   - Computationally expensive for large datasets

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No:92301733021**

Program (Code):

To be attached with

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
ratings = pd.read_csv("ratings.csv")
# ================= USER-ITEM MATRIX =================
user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)
# ================= USER SIMILARITY =================
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)
# ================= RECOMMEND FUNCTION =================
def recommend_movies(user_id, top_n=5):
    if user_id not in user_item_matrix.index:
        return "User not found!"

    # Similar users
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]

    # Weighted ratings
    weighted_ratings = np.zeros(user_item_matrix.shape[1])

    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values

    # Normalize
    weighted_ratings /= similar_users.sum()

    # Get unseen movies
    user_rated = user_item_matrix.loc[user_id]
    unseen_movies = user_rated[user_rated == 0].index

    scores = list(zip(unseen_movies, weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)
```

```
    recommended_ids = [movie for movie, _ in scores[:top_n]]

    return recommended_ids
print(recommend_movies(user_id=1))
```

Results:

To be attached with

```
•••  [1200, 1610, 541, 589, 1036]
```

Observation:

The system recommends movies based on user similarity using ratings data. It creates a user-item matrix and calculates similarity between users using cosine similarity. Movies liked by similar users are recommended to the target user.

It was observed that the recommendations are personalized and depend on user behavior. However, if a user has very few ratings, the system may not give accurate results.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date:**	**Enrolment No:92301733021**

Post Lab Exercise:

1. **Are you satisfied with the results/ratings shown by your designed model? If yes, justify. If no, describe the probable reason for under performance of the system.**

   The results are satisfactory because the system recommends movies based on similar users' preferences, which gives personalized suggestions. The use of cosine similarity helps in finding users with similar tastes, so the recommendations are relevant.

   However, the performance may not always be perfect because of limited user data or fewer ratings. If a user has rated very few movies, the system may not generate accurate recommendations.

2. **Recommend top-10 "movies" using MovieLens 100K dataset (Link: https://grouplens.org/datasets/movielens/100k/). Attach the code with output. Also, write your observation for the same.**

   Code:

```
# MOVIELENS TOP-10 MOVIES (100k)

import pandas as pd
import urllib.request
import zipfile

# Download dataset
url = "https://files.grouplens.org/datasets/movielens/ml-100k.zip"
urllib.request.urlretrieve(url, "ml-100k.zip")

# Extract dataset
with zipfile.ZipFile("ml-100k.zip", 'r') as zip_ref:
    zip_ref.extractall()

# Load ratings
ratings = pd.read_csv("ml-100k/u.data", sep="\t",
```

```
                    names=["user_id", "movie_id", "rating", "timestamp"])

# Load movie titles
movies = pd.read_csv("ml-100k/u.item", sep="|", encoding="latin-1",
                     names=["movie_id", "title"] + [str(i) for i in range(22)])

# Merge datasets
data = pd.merge(ratings, movies, on="movie_id")

# Compute average rating and count
avg_rating = data.groupby("title")["rating"].mean()
count = data.groupby("title")["rating"].count()

# Combine both
movie_stats = pd.DataFrame({
    "average_rating": avg_rating,
    "rating_count": count
})

# Filter popular movies
movie_stats = movie_stats[movie_stats["rating_count"] > 100]

# Get top 10 movies
top10 = movie_stats.sort_values(by="average_rating", ascending=False).head(10)

# Output
print("Top 10 Movies:\n")
print(top10)
```

Output:

```
Top 10 Movies:

                                  average_rating  rating_count
title
Close Shave, A (1995)                   4.491071           112
Schindler's List (1993)                 4.466443           298
Wrong Trousers, The (1993)              4.466102           118
Casablanca (1942)                       4.456790           243
Shawshank Redemption, The (1994)        4.445230           283
Rear Window (1954)                      4.387560           209
Usual Suspects, The (1995)              4.385768           267
Star Wars (1977)                        4.358491           583
12 Angry Men (1957)                     4.344000           125
Citizen Kane (1941)                     4.292929           198
```